Architecture	Host	Cores	Topology	Commit	`rustc`	Measured	Result set
`x86_64`	`a2`	64	4 sockets x 16 cores, 4 NUMA nodes	`cc90fa548767`	rustc 1.97.1 (8bab26f4f 2026-07-14)	2026-08-10	`x86_64/current`
`aarch64`	`galaxy`	128	1 socket x 128 cores, 1 NUMA node	`cc90fa548767`	rustc 1.97.1 (8bab26f4f 2026-07-14)	2026-08-10	`aarch64/current`

Table 1: The machines behind every other number in this document. A cross-architecture comparison is only meaningful if both sets measure the same commit; that column is here so a reader can confirm it rather than assume it.

Generated benchmark artifacts

Typeset from `paper/generated/cross-arch/` by `benchmarks/scripts/build_pdf.py`. Every number resolves to a row in the result set named below; see `PROVENANCE.md` for the inputs, transformation and caveats of each artifact.

`inputs: sha256:8b6b4f5e10ff4d20`

Benchmark	Metric	mapped	mapq 60	seeded	refined	reported	Verdict
B01	Containment	=	=	=	=	=	agree
B01	Jaccard	=	=	=	=	=	agree
B01	bucket_SH	=	=	=	=	=	agree
B02	Containment	=	=	=	=	=	agree
B02	Jaccard	=	=	=	=	=	agree
B02	bucket_SH	=	=	=	=	=	agree
B03	Containment	=	=	=	=	=	agree
B03	Jaccard	=	=	=	=	=	agree
B03	bucket_SH	=	=	=	=	=	agree
B04	Containment	=	=	=	=	=	agree
B04	Jaccard	=	=	=	=	=	agree
B04	bucket_SH	=	=	=	=	=	agree
B05	Containment	=	=	=	=	=	agree
B05	Jaccard	=	=	=	=	=	agree
B05	bucket_SH	=	=	=	=	=	agree

Table 2: Cross-architecture agreement of the algorithm's own counters. The mapper is deterministic, so for one commit these must be identical on every machine: they describe the computation, not its speed. Every cell reading = is what licenses the timing tables to be read as two views of one algorithm.

		x86_64			aarch64			aarch64/x86_64
Benchmark	Metric	index (s)	map (s)	RSS (GB)	index (s)	map (s)	RSS (GB)	map ×
B01	Containment	7.01	33.64	2.64	6.52	36.27	2.66	1.08
B01	Jaccard	7.18	50.97	2.60	6.50	50.77	2.65	1.00
B01	bucket_SH	6.93	25.00	2.62	6.53	27.77	2.63	1.11
B02	Containment	7.07	26.08	2.61	6.54	27.82	2.66	1.07
B02	Jaccard	6.99	38.30	2.61	6.50	43.53	2.65	1.14
B02	bucket_SH	7.01	21.62	2.58	6.54	23.97	2.66	1.11
B03	Containment	7.05	40.26	2.62	6.49	45.23	2.65	1.12
B03	Jaccard	7.03	52.80	2.61	6.53	61.01	2.65	1.16
B03	bucket_SH	6.97	31.54	2.58	6.51	36.23	2.65	1.15
B04	Containment	6.93	393.28	2.60	6.54	455.45	2.66	1.16
B04	Jaccard	6.97	534.67	2.60	6.52	622.92	2.65	1.17
B04	bucket_SH	7.01	308.62	2.63	6.51	360.70	2.65	1.17
B05	Containment	7.03	12.57	2.66	6.51	13.68	2.65	1.09
B05	Jaccard	6.91	14.94	2.67	6.54	16.80	2.66	1.12
B05	bucket_SH	6.93	11.12	2.61	6.49	12.57	2.65	1.13

Table 3: Indexing time, mapping time and peak memory on each machine, single-threaded. The -@1 column is the like-for-like one: it is the only thread count at which the machines are doing comparable amounts of work per unit of hardware.

Benchmark	Metric	`x86_64`	`aarch64`	Δ
B01	Containment	2.66×	2.39×	-0.28
B01	Jaccard	2.34×	2.26×	-0.07
B01	bucket_SH	2.88×	2.52×	-0.36
B02	Containment	2.76×	2.48×	-0.28
B02	Jaccard	2.61×	2.29×	-0.31
B02	bucket_SH	2.97×	2.59×	-0.38
B03	Containment	2.55×	2.16×	-0.39
B03	Jaccard	2.48×	2.10×	-0.37
B03	bucket_SH	2.62×	2.22×	-0.40
B04	Containment	2.22×	1.95×	-0.27
B04	Jaccard	2.14×	1.94×	-0.20
B04	bucket_SH	2.19×	1.98×	-0.21
B05	Containment	2.84×	2.51×	-0.32
B05	Jaccard	2.74×	2.37×	-0.37
B05	bucket_SH	2.89×	2.59×	-0.30

The C++ rows for `x86_64` (2026-08-10), `aarch64` (2026-08-10) were carried forward from an earlier run, so that column divides two measurement days and its cancellation of machine speed is only approximate.

Table 4: shmap-rs against the C++ `shmap`, measured separately on each machine. Unlike every other timing table here, this one is not confounded by the hosts differing: both terms of each ratio come from the same machine, so its speed cancels and what remains is a property of the two programs.

		Δ pp on `aarch64`				
Stage	`x86_64` share	B01	B02	B03	B04	B05
index: reading	9.7%	-3.6	-4.3	-3.9	-0.6	-6.5
index: sketching	12.4%	-0.1	+0.5	+0.0	-0.1	+1.2
index: collecting	7.6%	+1.1	+0.9	+0.3	+0.0	+1.9
index: finalizing	0.3%	-0.1	-0.1	-0.1	+0.0	-0.1
map: sketching	14.2%	+0.1	+1.4	+0.2	-0.6	+1.5
map: prepare	13.9%	+2.2	+2.2	+1.0	+1.0	+2.5
map: match_seeds	17.9%	+1.0	+1.7	+3.3	+2.6	+0.8
map: bucket_merge	2.8%	+0.0	+0.2	+0.1	-0.2	+0.0
map: match_rest	21.0%	-0.7	-2.5	-0.9	-2.2	-1.4
map: other (unattributed)	0.2%	+0.0	+0.1	+0.1	+0.1	+0.1

Table 5: Whether the shape of the run changes with the machine. Each stage's share of total CPU time on the reference architecture, and how many percentage points that share moves elsewhere. Shares, not seconds: seconds differ because the machines differ in speed, which says nothing; a moved share says a stage became relatively dearer.

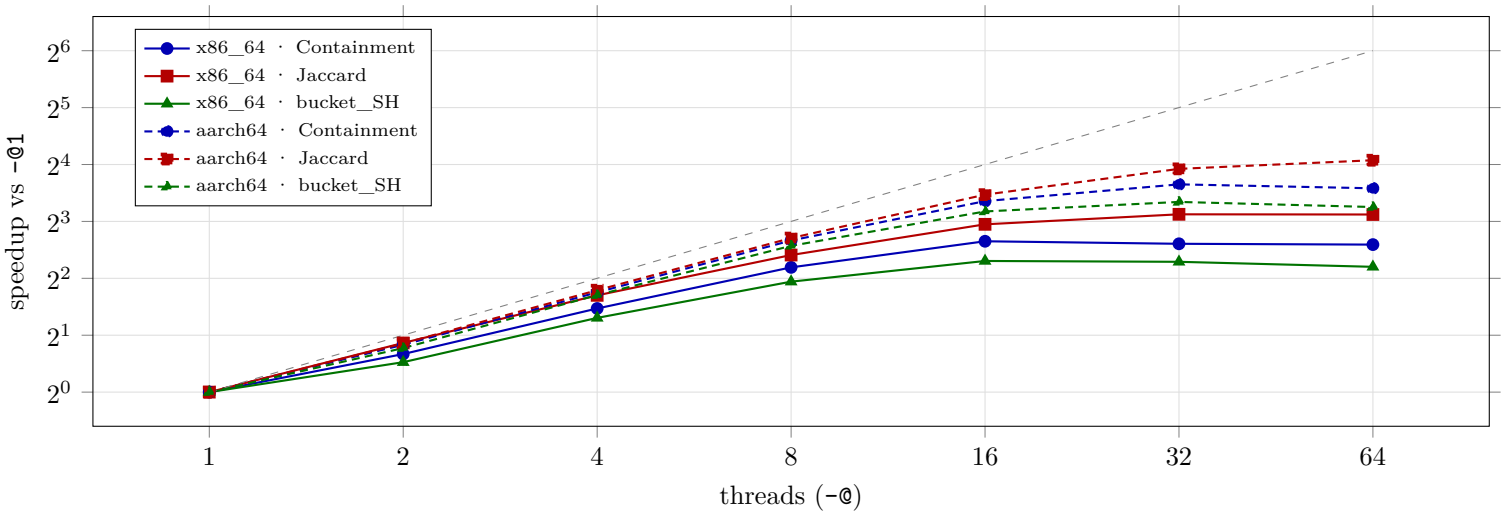

Figure 1: Thread scaling on each machine, benchmark B01. Colour is the metric, line style the architecture; the grey diagonal is linear speedup. Each curve is normalised to its own machine's single-threaded time, so the figure compares how well the machines scale, not how fast they are.

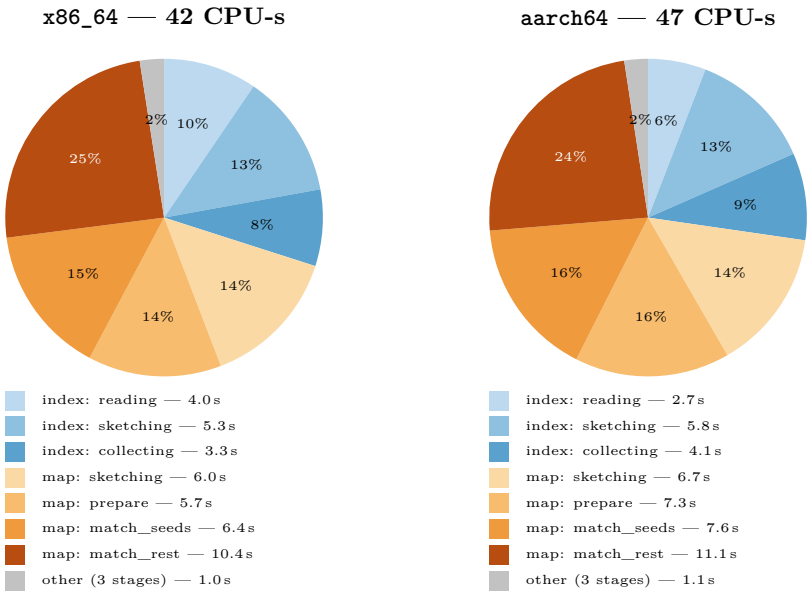

Figure 2: Where the CPU time goes on each machine, benchmark B01 (Containment, -@1). Indexing is blue, mapping orange, and wedges under 5% are folded into a single grey 'other'. Read against Table 5, which carries every benchmark.